DuckDB CSV Parsing Options

DuckDB features one of the most powerful and highly optimized CSV parsers available. It automatically detects schema configurations via a built-in "sniffer" using the read_csv function. However, when dealing with semi-structured, dirty, or non-standard delimited files, you can manually override this behavior using the parameters below.

1. Dialect & Structural Options

Use these options to define physical file properties, character definitions, and boundaries.

Parameter	Type	Default	Description
delim / **sep**	VARCHAR	','	The column separator. Can be a single character or multi-byte string (e.g., tabs '\t', pipes '|', or emojis like '🦆').
header	BOOL	*Auto*	Set to true to treat the first non-skipped row as column names, or false to treat it as data.
skip	BIGINT	0	The number of lines at the very top of the file to ignore before looking for headers or data rows.
quote	VARCHAR	'"'	The quoting character used to wrap fields containing delimiters, comments, or newlines.
escape	VARCHAR	'"'	The character used to escape a quote character inside a quoted string.
comment	VARCHAR	''	Any line beginning with this prefix is ignored entirely by the parser. Useful for inline notes or metadata.
new_line	VARCHAR	*Auto*	Explicitly define line endings: '\n', '\r', or '\r\n'.

```
-- Example: Reading a tab-separated file with metadata at the top
SELECT * FROM read_csv('raw_output.tsv',
    delim='\t',
    skip=5,
    comment='#'
);
```

2. Schema Enforcements & Type Guessing

Semi-structured data can often trick automatic type detection. These parameters give you exact control over schema validation.

Parameter	Type	Default	Description
all_varchar	BOOL	false	Skips type detection entirely. Loads **all** columns as text strings (VARCHAR). Highly recommended for unstable or deeply malformed files.
columns / **types**	STRUCT	*None*	Explicitly maps columns to target data types. Can be specified as a list ['INT', 'VARCHAR'] or a structural map {'id': 'INT', 'price': 'DOUBLE'}.
names	LIST	*None*	Overrides the column headers with a custom list of names, e.g., names=['col_a', 'col_b', 'col_c'].
sample_size	BIGINT	20480	Number of sample lines scanned to guess data types. Set to -1 to scan the **entire file** before resolving types.
auto_type_candidates	LIST	*Auto*	A custom list of SQL types that the automatic sniffer is allowed to consider (e.g., ['BIGINT', 'VARCHAR', 'DATE']).

```
-- Example: Explicitly mapping types and forcing a full-file type scan
SELECT * FROM read_csv('users.csv',
    types={'id': 'BIGINT', 'joined_on': 'DATE'},
    sample_size=-1
);
```

3. Handling Errors, Footers, & Malformed Rows

For files that contain dynamic column counts, trailing notes, summary footers, or unparseable rows.

Parameter	Type	Default	Description
ignore_errors	BOOL	false	If true, any rows that fail parsing or contain mismatched types are silently ignored instead of throwing an execution error.
null_padding	BOOL	false	When a row contains fewer columns than defined by the schema, pads the missing rightmost columns with NULL instead of failing.

strict_mode	BOOL	true	When set to false, the parser tolerates unescaped quotes, inconsistent quoting, and minor column count drift.
nullstr / **null**	LIST	['']	A list of explicit text strings that should be cast strictly as a SQL NULL value (e.g., nullstr=['NA', 'NaN', 'NULL', '-']).

```
-- Example: Robustly reading highly unstructured files by skipping corrupt rows
SELECT * FROM read_csv('dirty_records.txt',
    all_varchar=true,
    ignore_errors=true,
    null_padding=true
);
```

4. Date, Time, & Decimal Formatting

These parameters override standard ISO-8601 formatting assumptions.

Parameter	Type	Default	Description
dateformat	VARCHAR	*Auto*	Specifies custom formatting templates to parse date columns (e.g., '%d-%m-%Y' or '%b %d, %Y').
timestampformat	VARCHAR	*Auto*	Specifies custom formatting templates for timestamp columns (e.g., '%Y/%m/%d %H:%M:%S.%f').
decimal_separator	VARCHAR	'.'	Modifies the symbol used to define decimal points (change to ',' for European style formatting).

```
-- Example: European formatting with customized timestamp syntax
SELECT * FROM read_csv('european_sales.csv',
    decimal_separator=',',
    timestampformat='%d.%m.%Y %H:%M:%S'
);
```

5. Multi-File & Partitioning Management

Useful options when scanning directories, patterns, or buckets containing many similar files at once.

Parameter	Type	Default	Description

filename	BOOL	false	Appends a virtual column containing the exact path/filename from which each row was sourced.
hive_partitioning	BOOL	*Auto*	Automatically translates subdirectory structures (e.g., /year=2026/month=06/) into dynamic columns.
union_by_name	BOOL	false	Resolves mismatched column counts or orders across files by matching columns by name rather than physical position, introducing NULL for missing elements.

```
-- Example: Unioning directories with shifting schemas while tracking file origins
SELECT filename, status, count(*)
FROM read_csv('logs/*/*.csv',
    filename=true,
    union_by_name=true
)
GROUP BY filename, status;
```

Practical Recipes for Semi-Structured Files

Recipe A: The "Total Chaos" File

Situation: The file has dynamic columns, random lines of comments, mismatched quotes, and bad values that trigger type cast errors.

```
SELECT * FROM read_csv('unstable_log.txt',
    all_varchar=true,      -- Safe reading
    ignore_errors=true,    -- Drop rows that won't parse
    null_padding=true,     -- Fill in trailing missing values
    strict_mode=false,     -- Relax quote validation
    comment='#'            -- Ignore lines beginning with '#'
);
```

Recipe B: Header Skip & Exclude "Start/Stop" Tags

Situation: File contains a metadata head, and has lines starting with structural indicators "Start" and "Stop" inside the raw data.

```
-- Force all column structures as text, filter, and cast manually
SELECT
    column_0::INTEGER as record_id,
    column_1 as event_name,
```

```
    column_2::DOUBLE as latency
FROM read_csv('process_dump.log',
    skip=10,              -- Bypasses 10 metadata header lines at top
    all_varchar=true,     -- Avoids casting errors on "Start" / "Stop" lines
    header=false          -- No direct headers (we name them manually)
)
WHERE column_0 NOT LIKE 'Start%'
  AND column_0 NOT LIKE 'Stop%';
```

Recipe C: Deep Audit Using sniff_csv

If you are struggling to parse a file, ask DuckDB what configuration it *thinks* is right before executing.

```
-- Inspect the sniffer metadata output
FROM sniff_csv('mystery_file.csv');
```

DuckDB Dynamic CSV Parser Configuration

```yaml
# Use the following to programmatically map incoming file profiles to DuckDB's read_csv parameters.


# ========================================

# MASTER SCHEMA SPECIFICATION & DEFAULT VALUES

# ========================================

default_csv_options:

  # Dialect & Structure

  delim: ","                  # varchar (e.g., ',', '\t', '|')

  header: true                # boolean
```

```
  skip: 0                     # integer (lines to skip at top)
  quote: '"'                  # varchar (quoting character)
  escape: '"'                 # varchar (escape character)
  comment: ""                 # varchar (ignore lines starting with this prefix)
  new_line: "auto"            # varchar ('\n', '\r', '\r\n', or 'auto')

  # Schema Control
  all_varchar: false          # boolean (if true, forces all columns to VARCHAR)
  sample_size: 20480          # integer (number of lines to sample, -1 for entire
file)
  columns: {}                 # map of col_name: type (e.g., id: INT, date: DATE)
  names: []                   # list of custom column headers (overrides file
headers)

  # Error & Messy Data Handling
  ignore_errors: false        # boolean (skip malformed/uncastable rows)
  null_padding: false         # boolean (pad missing rightmost elements with
NULL)
  strict_mode: true           # boolean (disable to tolerate quote/column drifts)
  nullstr: [""]               # list of strings to translate to SQL NULL

  # Date & Format Specifics
```

```
  dateformat: "auto"           # varchar (e.g., '%d-%m-%Y')
  timestampformat: "auto"      # varchar (e.g., '%Y-%m-%d %H:%M:%S.%f')
  decimal_separator: "."       # varchar ('.' or ',')

  # Multi-File Management
  filename: false              # boolean (inject source filename column)
  hive_partitioning: false     # boolean (extract directory structures as columns)
  union_by_name: false         # boolean (match columns across files by header name)

  # Custom Post-Processing Rules (for programmatic WHERE clause generation)
  exclude_prefixes: []         # list of string prefixes to filter out of the target column
  exclude_target_column: ""    # column index or name to apply the exclude_prefixes filter on

# ========================================
# CONCRETE PROFILES FOR DELIMITED FILE TYPES
# ========================================
file_profiles:

  # TYPE 1: Standard, Clean CSV Imports
```

```yaml
  standard_csv:
    description: "Standard well-formed CSV documents"
    delim: ","
    header: true
    strict_mode: true

  # TYPE 2: The "Total Chaos" Raw Logger
  # Handles unequal columns, stray quotes, and casting failures gracefully
  unstable_log:
    description: "System logs with dynamic schemas, unmatched quotes, and corrupt rows"
    all_varchar: true
    ignore_errors: true
    null_padding: true
    strict_mode: false
    comment: "#"
    nullstr: ["", "NULL", "NaN", "N/A", "-"]

  # TYPE 3: Metadata Header Files with Structural Blocks (e.g., Start/Stop tags)
  # Uses post-processing directives to strip metadata and start/stop indicators
  metadata_header_stream:
    description: "Files containing legal metadata at the top and system markers"
```

```
    skip: 10
    header: false
    all_varchar: true
    exclude_target_column: "column_0"
    exclude_prefixes: ["Start", "Stop", "Divider---"]
    columns:
      column_0: "VARCHAR"
      column_1: "VARCHAR"
      column_2: "VARCHAR"


  # TYPE 4: European Financial or Industrial Log
  # Semicolon delimited, commas for decimals, European date stamps
  european_financial:
    description: "EU formatted exports using commas for decimals and semicolons as separators"
    delim: ";"
    decimal_separator: ","
    dateformat: "%d.%m.%Y"
    timestampformat: "%d.%m.%Y %H:%M:%S"
    sample_size: -1 # Scan full file to avoid type mismatches on large financial lists
```

```yaml
# TYPE 5: Multi-File Partitioned Directory Scan
# Seamlessly unifies multi-file directories with minor schema updates over time
partitioned_historical_log:
  description: "Bulk historical analysis spanning directories with hive layout"
  filename: true
  hive_partitioning: true
  union_by_name: true
  ignore_errors: true
```